Module 01 — Functions, Decorators & Generators

Phase: Advanced Python
Previous module: Data Structures Overview
Next module: Modules and Packages

Module Outcome:

- User-defined functions (`def`, `return`)
- Types of arguments (positional, keyword, default, `*args` / `**kwargs`)
- Scope — local vs global (LEGB)
- Lambda, `map` / `filter` / `reduce`
- Decorators
- Generators

Motivation — Feel the pain of copy-paste

Look at this ATM-style script. The same `amount <= 0` check appears in every action:

```
balance = 1000.0

# --- deposit ---
amount = 200.0
if amount <= 0:
    print("Invalid amount")
else:
    balance = balance + amount
    print("Deposited. Balance:", balance)

# --- withdraw ---
amount = 150.0
if amount <= 0:
    print("Invalid amount")
else:
    if amount > balance:
        print("Insufficient funds")
    else:
        balance = balance - amount
        print("Withdrawn. Balance:", balance)

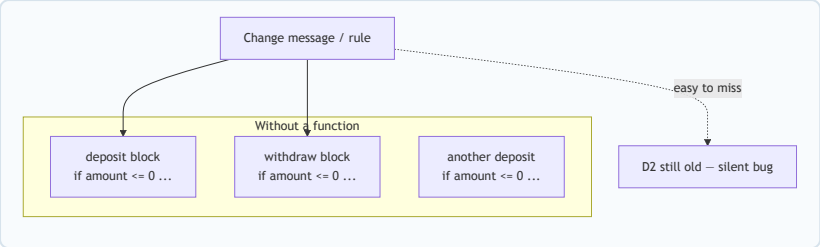
# --- another deposit (copy-paste again) ---
amount = -50.0
if amount <= 0:
    print("Invalid amount")
else:
    balance = balance + amount
    print("Deposited. Balance:", balance)
```

Problems with this style:

1. The same validation is written **three times**.
2. Changing the message to `"Amount must be positive"` means editing **every** copy.

3. Miss one copy → one path keeps the old behaviour (a silent bug).

Visual — same logic, three copies (the bug factory):



Key takeaways:
- Copy-paste the same check → bug factory when rules change
- One missed copy keeps the old behaviour silently
- Fix: write the rule once in a function, call it everywhere

Write the check **once**, then reuse it:

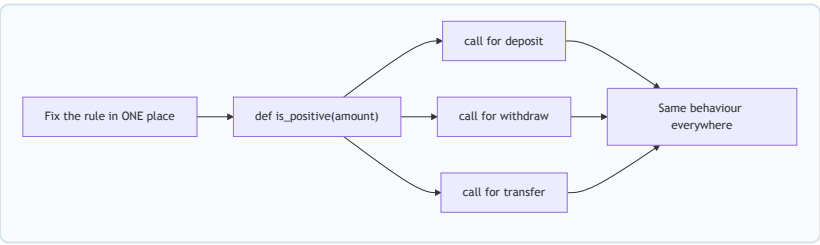
```
def is_positive(amount):
    if amount <= 0:
        print("Invalid amount")
        return False
    return True

balance = 1000.0

if is_positive(200.0):
    balance = balance + 200.0

if is_positive(-50.0):
    balance = balance + (-50.0)  # never runs — guard stopped us
```

Visual — write once, call many times:



Key takeaways:
- `def` names the reusable unit
- Many call sites → one definition
- Change the function once → all callers update

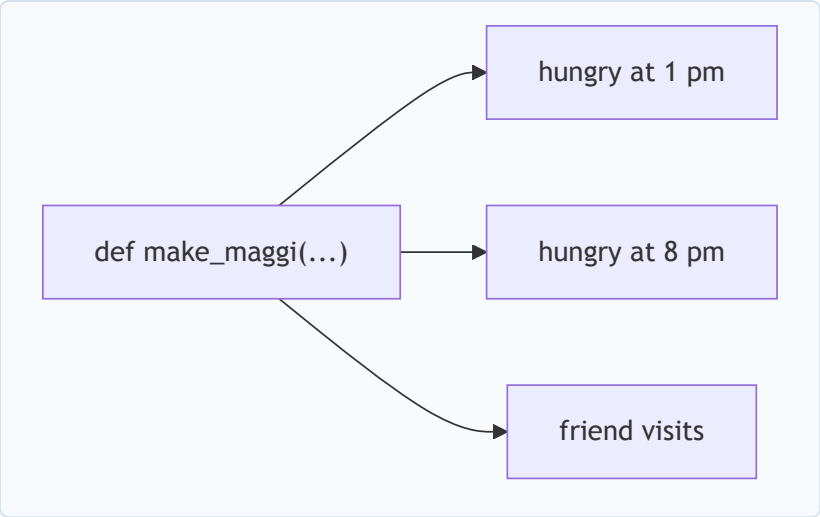
That named unit is a **function**. Your lab kit (Refactor ATM into functions) is the same idea at ATM scale.

What is a function?

If a group of statements is needed again and again, do **not** copy-paste them. Bundle those statements into **one named unit**, then **call** that unit whenever you need it. That unit is a **function**.

Kitchen analogy: instead of rewriting "boil water → noodles → masala" every time, you write the recipe once (`def make_maggi(...)`) and call it when hungry.

Visual — recipe once, cook many times:



Key takeaways:
- Recipe once = function once
- Hunger / call sites can differ; the steps stay the same
- Name the function after the job (`make_maggi`, not `f1`)

In other languages the same idea may be called a *method*, *procedure*, or *subroutine*. In Python we mostly say **function** (and later **method** when it lives on a class).

Why do we need functions?

Without functions	With functions
Same validation copied in 5 places	One `is_valid_email()` reused everywhere
Bug fixed in one place, missed in others	Fix once — every call benefits
400-line script hard to read	Named steps: `load()`, `process()`, `save()`

Main advantage: code reusability — write once, call many times. Secondary wins: clearer names, easier testing, and smaller pieces you can put in modules later.

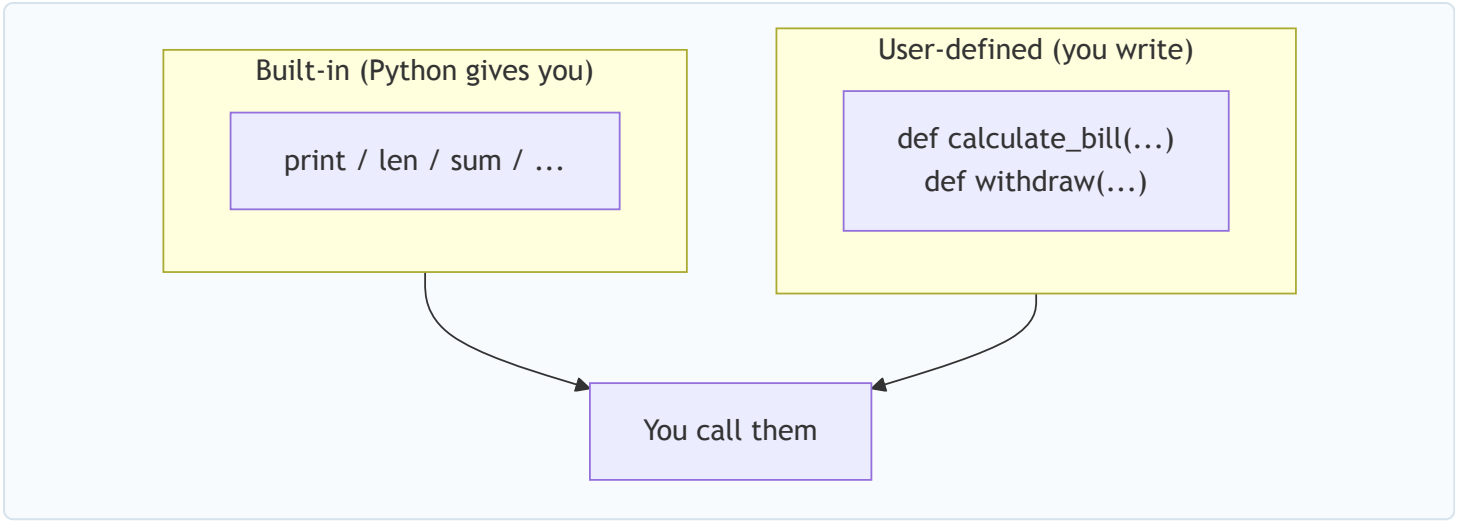
🏦 Bank 📱 UPI 📦 Order 💰 Payroll 💻 Laptop

Where used?
- **SecureBank** / UPI — validate amount once, reuse in deposit / withdraw / transfer
- **TechMart** — GST map, cart totals, login-gated checkout helpers
- **AcmeHR** — bill / salary calculators with defaults and `*args` for allowances
- Reporting — generators stream large attendance or expense rows without loading everything
- Toolkits — shared `ksr_utils` validators and formatters imported across scripts

Two kinds of functions in Python

1. **Built-in (predefined)** — already available with Python: `print()`, `len()`, `type()`, `input()`, `int()`, `sum()`, ...
2. **User-defined** — you write them for your project: `calculate_bill()`, `wish()`, `withdraw()`.

Visual — you already call built-ins; now you create your own:



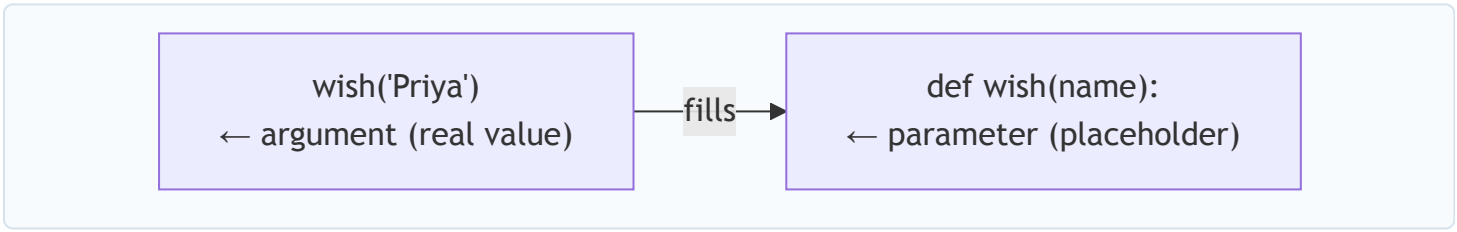
You already *use* built-ins. This module teaches you to *create* user-defined ones, then stretch into decorators and generators.

Parameter vs argument (words interviewers mix up)

- **Parameter** — placeholder name in the definition: `def wish(name):`
- **Argument** — real value you pass when calling: `wish("Priya")`

Recipe placeholders vs the ingredients you actually put in the bowl.

Visual — definition vs call:



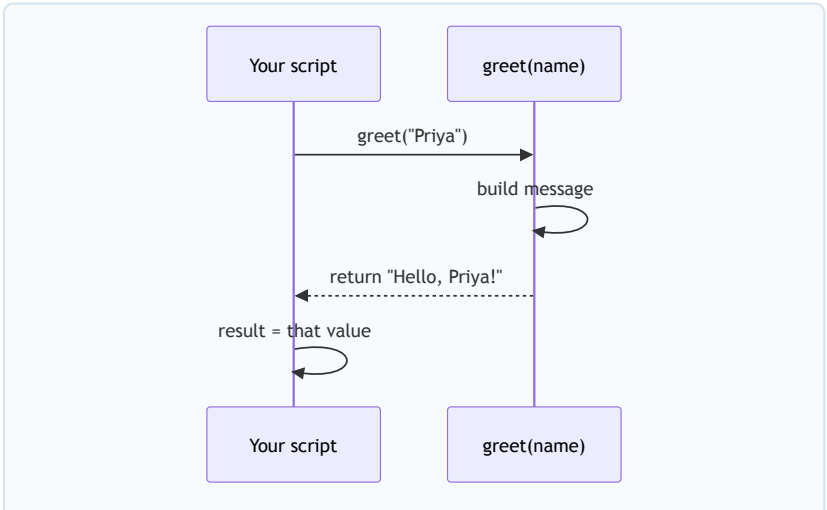
1. Creating a user-defined function (`def` and `return`)

Definition: A named block that runs when called.
Purpose: So you can reuse logic and give that logic a clear name.

Two keywords matter:

1. `def` — **mandatory** — starts the definition
2. `return` — **optional** — sends a result back to the caller (if omitted, Python returns `None`)

Visual — call → run body → return to caller:



Key takeaways:
- Call runs the body; `return` sends a value back
- No `return` → caller gets `None`
- Capture the result: `result = greet("Priya")`

```
def wish():
    """Print a greeting. No inputs, no return value."""
    print("Hello Good Morning")

wish()
wish()  # call as many times as you need — no rewrite
```

With an input (parameter) and a returned result:

```
def greet(name):          # name is a parameter
    message = f"Hello, {name}!"
    return message        # sends value back to caller

result = greet("Priya")   # "Priya" is the argument
print(result)             # Hello, Priya!
```

Expected Output:

```
Hello, Priya!
```

Rules people often forget:

- `return` exits the function immediately (code after it does not run).
- No `return` → the call evaluates to `None` (easy to miss when you `print(f())`).
- Python can return **multiple values** as a tuple: `return sum, diff` then `x, y = sum_sub(10, 5)`.

Remember: No `return` → the call evaluates to `None`. Printing `print(f())` when you meant to return a value is a classic silent bug.

Write a function that converts Celsius to Fahrenheit and print the result for 0°C, 37°C, and 100°C.

```
def celsius_to_fahrenheit(c):
    return c * 9 / 5 + 32

for temp in (0, 37, 100):
    print(f"{temp}°C → {celsius_to_fahrenheit(temp)}°F")
```

Expected Output:

```
0°C → 32.0°F
37°C → 98.6°F
100°C → 212.0°F
```

Write a function that returns `"Even"` or `"Odd"` and print the result for 10 and 15.

```
def even_odd(n):
    return "Even" if n % 2 == 0 else "Odd"

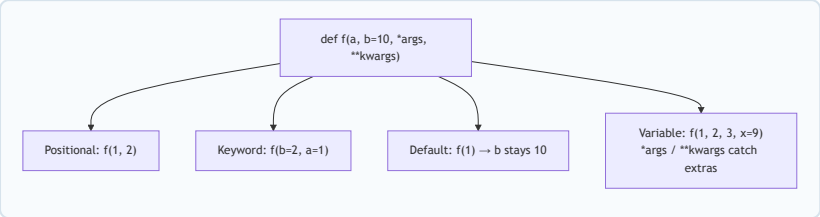
print(even_odd(10))  # Even
print(even_odd(15))  # Odd
```

2. Types of arguments (what / why, then syntax)

When you write `def sub(a, b):` and call `sub(100, 200)`, `a` / `b` are **formal** parameters and `100` / `200` are **actual** arguments.

Python allows four common ways to pass arguments. Understand **why** each exists before memorizing the table.

Visual — four ways to fill the same recipe:



Key takeaways:
- Positional = by order; keyword = by name
- Defaults make optional params
- `*args` / `**kwargs` catch extras (tuple / dict)

2a. Positional arguments

Definition: Values match parameters **by order**.
Purpose: Simple and short for small APIs.
Caution: Swap order → wrong result (`sub(100, 200)` ≠ `sub(200, 100)`). Count must match.

```
def sub(a, b):
    return a - b

print(sub(100, 200))  # -100
print(sub(200, 100))  # 100
```

2b. Keyword arguments

Definition: Pass by **parameter name**: `wish(name="Ravi", msg="Hi")`.
Purpose: Order no longer matters; calls stay readable when there are many options.
Rule: After a keyword argument, you cannot put a bare positional one.

```
def wish(name, msg):
    print(f"Hello {name}, {msg}")

wish(name="Ravi", msg="Good Morning")
wish(msg="Good Morning", name="Ravi")  # same result
# wish(name="Ravi", "Good Morning")  # SyntaxError
```

2c. Default arguments

Definition: A parameter already has a value if the caller skips it.
Purpose: Optional behaviour without writing two functions (`wish("Ravi")` and `wish()`).
Rule: Non-default parameters must come **before** defaults (`def wish(name="Guest", msg):` is invalid).

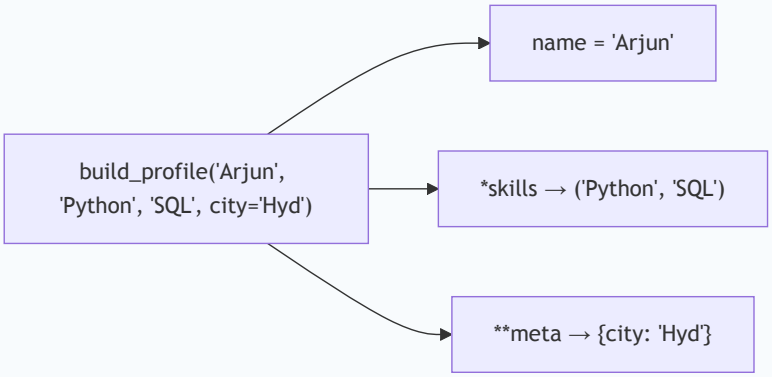
```
def wish(name="Guest"):
    print(f"Hello {name}, Good Morning")

wish("Ravi")
wish()  # uses "Guest"
```

2d. Variable-length arguments (`*args`, `**kwargs`)

Definition: Accept **any number** of positional (`*args` → tuple) or keyword (`**kwargs` → dict) values.
Purpose: Helpers like `sum_all(...)`, flexible loggers, and wrappers that forward arguments.
Rule (definition order): required positional → `*args` → keyword-only → `**kwargs`.

Visual — packing extras into a bag:



Key takeaways:
- Required params first, then `*args`, then `**kwargs`
- `*args` is a tuple of leftover positionals
- `**kwargs` is a dict of leftover keywords

```
def sum_all(*numbers):
    total = 0
    for n in numbers:
        total += n
    return total

print(sum_all())           # 0
print(sum_all(10, 20, 30)) # 60

def build_profile(name, *skills, **meta):
    return {"name": name, "skills": list(skills), **meta}

print(build_profile("Arjun", "Python", "SQL", city="Hyderabad"))
```

Quick reference (after you understand the why):

Type	Syntax	Use when
Positional	`f(a, b)`	Few args; order is obvious
Keyword	`f(b=5, a=2)`	Clarity / optional skip of order
Default	`def f(a, b=10)`	Sensible optional value
`*args`	`def f(*args)`	Unknown count of positional values
`**kwargs`	`def f(**kwargs)`	Unknown named options

Write `greet` with a default message of `"Good Morning"`. Call it once with a positional name, then once with keyword arguments for name and a custom message.

```
def greet(name, msg="Good Morning"):
    print(f"Hello {name}, {msg}")

greet("Priya")
greet(name="Ravi", msg="Good Evening")
```

Expected Output:

```
Hello Priya, Good Morning
Hello Ravi, Good Evening
```

Write `total(*amounts, tax_rate=0.05)` that sums the amounts, adds tax, rounds to 2 decimals, and print `total(100, 200, 50)`.

```
def total(*amounts, tax_rate=0.05):
    subtotal = sum(amounts)
    return round(subtotal * (1 + tax_rate), 2)

print(total(100, 200, 50))  # 367.5
```

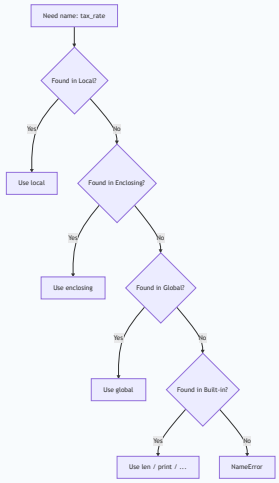
3. Scope — local vs global (and LEGB)

Definition: Where a name is visible.
Purpose: Without scope rules, every variable would collide; with them, functions stay self-contained.

- **Global** — declared outside functions; readable from functions in that module.
- **Local** — declared inside a function; invisible outside.

Python looks up names in this order (**LEGB**): **L**ocal → **E**nclosing (nested `def`) → **G**lobal → **B**uilt-in (`len`, `print`, …).

Visual — name lookup stops at the first hit:



Key takeaways:
- LEGB: Local → Enclosing → Global → Built-in
- Reading a global is fine; reassigning needs `global`
- Prefer pass-in / return-out over `global`

```
tax_rate = 0.18  # global

def calculate_total(amount):
    tax = amount * tax_rate   # read global OK
    return amount + tax
```

Use `global x` only when you must **reassign** a global name. Prefer passing values in and returning results — globals make bugs hard to find in real projects.

Common Error: Assigning to a name inside a function makes it local for the whole function — `x = x + 1` without `global` raises `UnboundLocalError`.

Bump a counter by returning `n + 1` and assigning the result back — do not use `global`. Print the updated count.

```
count = 0

def bump(n):
    return n + 1

count = bump(count)
print(count)  # 1
```

Write `make_counter()` that returns a nested function using `nonlocal` so two calls print 1 then 2 with no globals.

```
def make_counter():
    n = 0

    def next_count():
        nonlocal n
        n += 1
        return n

    return next_count

c = make_counter()
print(c())  # 1
print(c())  # 2
```

4. Lambda (anonymous) functions

Definition: A tiny function **without a** `def` **name**, written as `lambda args: expression`.
Purpose: Instant one-off use — especially as the `key=` for `sorted`, or with `map`/`filter`.
When not to: Multi-step logic, docstrings, or anything you will reuse and test — use `def`.

```
square = lambda n: n * n
print(square(5))  # 25

names = ["Meera", "Ravi", "An"]
print(sorted(names, key=lambda x: len(x)))
```

Lambda **returns** the expression result automatically — no `return` keyword.

Sort `["Meera", "Ravi", "An"]` by string length using a `lambda` key and print the sorted list.

```
names = ["Meera", "Ravi", "An"]
print(sorted(names, key=lambda x: len(x)))  # ['An', 'Ravi', 'Meera']
```

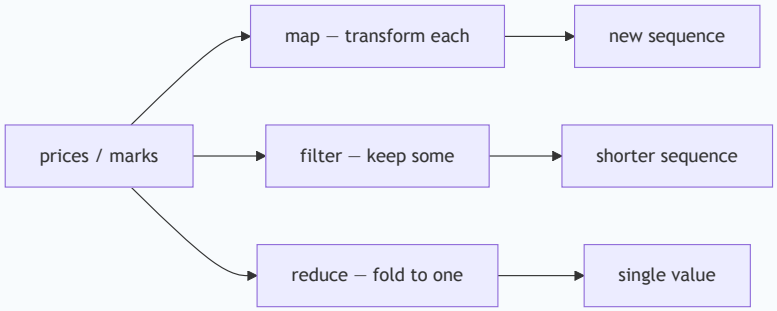
Sort a list of `(name, marks)` tuples by marks descending using a `lambda` key and print the result.

```
marks = [("A", 88), ("B", 95), ("C", 70)]
print(sorted(marks, key=lambda row: row[1], reverse=True))
# [('B', 95), ('A', 88), ('C', 70)]
```

5. `map`, `filter`, and `reduce`

Definition: Built-in helpers that apply a function across an iterable.
Purpose: Short pipelines for “transform every item”, “keep matching items”, or “fold many values into one” — often with `lambda` or a small `def`.

Visual — three jobs on a list of numbers:



Key takeaways:
- `map` transforms each item
- `filter` keeps matching items
- `reduce` folds many → one (prefer `sum` for plain totals)

Tool	Idea	Campus example
`map(fn, data)`	Transform each item	Prices → prices with GST
`filter(fn, data)`	Keep items where `fn` is true	Marks ≥ 75 only
`reduce(fn, data[, start])`	Combine into one value	Sum / max of expenses

`map` and `filter` return **lazy iterators** in Python 3 — wrap with `list(...)` when you need a list. Prefer a **list comprehension** when it reads clearer; still learn these for interviews and older codebases.

```
from functools import reduce

prices = [100, 250, 80]
with_gst = list(map(lambda p: round(p * 1.05, 2), prices))
print(with_gst)  # [105.0, 262.5, 84.0]

marks = [45, 78, 92, 61, 88]
passed = list(filter(lambda m: m >= 75, marks))
print(passed)  # [78, 92, 88]

total = reduce(lambda acc, x: acc + x, prices, 0)
print(total)  # 430
```

Same ideas with comprehensions (often clearer):

```
with_gst = [round(p * 1.05, 2) for p in prices]
passed = [m for m in marks if m >= 75]
```

Rules:

- `map` / `filter` need a **function** (named or `lambda`) that takes one item (or more if you pass several iterables to `map`).
- `reduce` lives in `functools` (not a built-in). The function takes **(accumulator, next_item)**.
- Do not force `reduce` for a simple sum — `sum(prices)` is better. Use `reduce` when the fold is custom (e.g. merge dicts, running max with a twist).

For TechMart canteen prices, use `map` to add 5% GST, then `filter` for prices ≥ 50. Print both lists.

```
prices = [40, 25, 60, 15, 120]
with_gst = list(map(lambda p: round(p * 1.05, 2), prices))
big = list(filter(lambda p: p >= 50, with_gst))
print(with_gst)
print(big)
```

Expected Output:

```
[42.0, 26.25, 63.0, 15.75, 126.0]
[63.0, 126.0]
```

Use `functools.reduce` to total a TechMart bill, then print `sum(bill)` beside it to show they match.

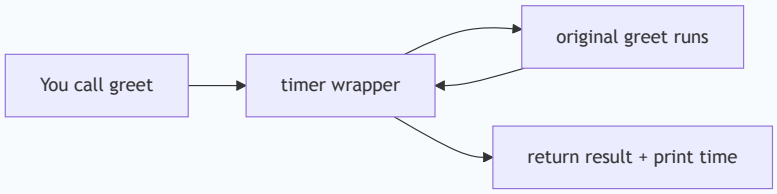
```
from functools import reduce

bill = [40, 25, 60]
total = reduce(lambda acc, x: acc + x, bill, 0)
print(total)       # 125
print(sum(bill))   # same — prefer sum for plain addition
```

6. Decorators

Definition: A function that takes another function and returns a **wrapped** version (extra behaviour before/after).
Purpose: Cross-cutting concerns — timing, login checks, retries — without copy-pasting into every function.

Visual — `@timer` wraps your function like gift wrap:



Key takeaways:
- `@timer` means `greet = timer(greet)`
- Wrapper runs before/after the original
- Great for login checks, timing, retries — without copy-paste

```
def timer(func):
    def wrapper(*args, **kwargs):
        # before logic
        result = func(*args, **kwargs)
        # after logic
        return result
    return wrapper

@timer
def slow_task():
    ...
```

`@timer` above `def slow_task` means: `slow_task = timer(slow_task)`.

Interview: A decorator is a function that takes a function and returns a wrapped function. `@dec` is sugar for `func = dec(func)`.

Write a `@shout` decorator that uppercases the wrapped function's return value and print `greet("Priya")`.

```
def shout(func):
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs).upper()
    return wrapper

@shout
def greet(name):
    return f"hello, {name}"

print(greet("Priya"))  # HELLO, PRIYA
```

Expected Output:

```
HELLO, PRIYA
```

Write `@require_login` that denies access when `current_user` is `None`, then allows `view_dashboard` after you set a user. Print both calls.

```
current_user = None

def require_login(func):
    def wrapper(*args, **kwargs):
        if current_user is None:
            print("Access denied — please log in")
            return None
        return func(*args, **kwargs)
    return wrapper

@require_login
def view_dashboard():
    return f"Welcome, {current_user}"

print(view_dashboard())   # Access denied
current_user = "Ravi"
print(view_dashboard())   # Welcome, Ravi
```

Expected Output:

```
Access denied — please log in
None
Welcome, Ravi
```

7. Generators

Calendar: Session 31 — Generators vs lists.
Lab kit: 15_generator_csv_stream (in-memory `yield` now; real CSV after Files week).

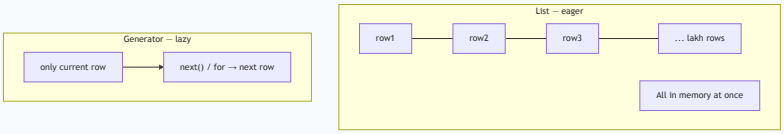
7a. List vs generator (Definition / Purpose + table)

Definition: A **generator** is an iterator that produces values **lazily** — one at a time — using `yield` inside a function, or a **generator expression** `(… for … in …)`.

Purpose: A list of lakhs of rows holds everything in RAM at once. A generator holds **only the current value** (plus a little state). Use it for streams, large reports, and “process then forget” pipelines.

Kitchen analogy: a list is the whole buffet on the table; a generator is a waiter who brings **one dish at a time**.

Visual — RAM picture (buffet vs waiter):



Key takeaways:
- List = whole buffet in RAM (eager)
- Generator = waiter brings one dish (lazy)
- Use generators for large one-pass streams

	List / list comprehension	Generator / `yield`
When built	Immediately (eager)	On demand (lazy)
Memory	All items	One item at a time
Syntax	`[x*2 for x in range(n)]`	`(x*2 for x in range(n))` or `yield`
Reusable	Yes — walk it many times	**Exhausted** after one full pass
Indexing	`data[0]` works	`gen[0]` fails — use `next` / `for`
Best for	Small/medium data you reuse	Large streams, one-pass processing

```
# Eager — builds the whole list in memory first
squares_list = [x * x for x in range(5)]
print(squares_list)  # [0, 1, 4, 9, 16]

# Lazy — nothing computed until you iterate
squares_gen = (x * x for x in range(5))
print(squares_gen)          # <generator object ...>
print(list(squares_gen))    # [0, 1, 4, 9, 16] — now exhausted
```

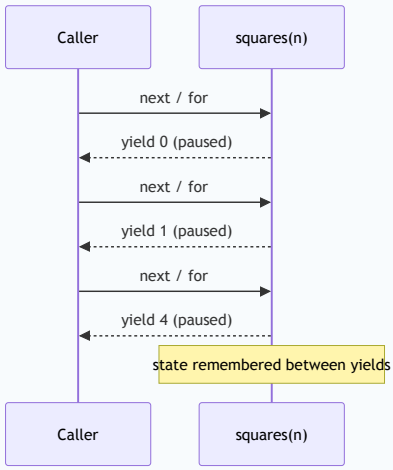
Note:

- Calling a generator function does **not** run the body yet — it returns a generator object. Values appear only when you iterate or call `next()`.

7b. `yield` vs `return`

	`return`	`yield`
Ends the function?	Yes (exits)	Pauses — resumes on next request
Gives back	One value (or `None`)	One value **per pause**; many over time
After call	Function is done	Generator object keeps state

Visual — `return` exits; `yield` pauses and resumes:




```
def one_square(n):
    return n * n          # function ends here

def squares(n):
    for x in range(n):
        yield x * x       # pause; resume for next x
```

7c. `next()`, loops, and exhaustion

Visual — one pass empties the generator:



Key takeaways:
- One full pass → generator is empty
- Extra `next` → `StopIteration`
- Need another pass? Call the generator function again

```
def countdown(n):
    while n >= 1:
        yield n
        n -= 1

g = countdown(3)
print(next(g))  # 3
print(next(g))  # 2
print(next(g))  # 1
# print(next(g))  # StopIteration — generator is exhausted

# Prefer a for-loop in real code (handles StopIteration for you)
for value in countdown(3):
    print(value, end=" ")  # 3 2 1
print()

# One full pass — then empty
g2 = countdown(2)
print(list(g2))  # [2, 1]
print(list(g2))  # []  — already exhausted; create a new generator if you need another pass
```

Note:

- Do not index a generator like a list (`gen[0]` fails). Use `next(gen)` or a `for` loop.

- After one full `for` / `list(gen)`, the generator is empty. Call the function again to get a fresh generator.

Write a `countdown` generator. Pull the first three values with `next`, then loop the rest.

```
def countdown(n):
    while n >= 1:
        yield n
        n -= 1

g = countdown(5)
print(next(g), next(g), next(g))  # 5 4 3
for value in g:
    print(value, end=" ")  # 2 1 — remaining values only
print()
```

Expected Output:

```
5 4 3
2 1
```

Write `even_stream(limit)` that yields even numbers from 0 through the limit and print `list(even_stream(10))`.

```
def even_stream(limit):
    n = 0
    while n <= limit:
        yield n
        n += 2

print(list(even_stream(10)))  # [0, 2, 4, 6, 8, 10]
```

7d. Generator expressions + when to use which

Same idea as a list comprehension, but with **parentheses** — lazy, not a list.

```
marks = [45, 78, 92, 33, 88]
# List — all at once
passed_list = [m for m in marks if m >= 40]
# Generator — one at a time
passed_gen = (m for m in marks if m >= 40)

print(sum(passed_gen))   # 303 — consumes the generator
# print(sum(passed_gen))  # 0 — already exhausted
```

When to use which:

- Need the full collection, reuse, or indexing → **list** comprehension.
- One-pass filter/transform (especially large) → **generator** expression or `yield` function.

Build a TechMart GST generator expression (5% on each price) and print each value one at a time.

```
prices = [40, 25, 60, 15, 120]
with_gst = (round(p * 1.05, 2) for p in prices)
for p in with_gst:
    print(p)
```

Expected Output:

```
42.0
26.25
63.0
15.75
126.0
```

Write a generator that yields marks below 40 and use `next(..., None)` so you stop at the first failing mark without scanning the whole list first.

```
marks = [80, 72, 35, 90, 28]

def failing(marks, cutoff=40):
    for m in marks:
        if m < cutoff:
            yield m

first = next(failing(marks), None)  # default None if none fail
print(first)  # 35 — did not need to scan the whole list into memory first
```

7e. Row-stream lab pattern + Done when

In class / homework: yield **dict rows from an in-memory list**. Opening CSV files comes in **Files week** (Stretch on the same lab kit).

```
SAMPLE_ROWS = [
    {"name": "Anita", "city": "Hyderabad"},
    {"name": "Ravi", "city": "Chennai"},
    {"name": "Priya", "city": "Bengaluru"},
]

def iter_rows(rows):
    for row in rows:
        yield row

for i, row in enumerate(iter_rows(SAMPLE_ROWS)):
    print(row)
    if i >= 1:
        break  # stop early — remaining rows never forced into a big result list
```

Lab Done when: 15_generator_csv_stream

- [ ] `iter_rows` uses `yield` (not `return` of a full list)
- [ ] Printing a few rows works without building an extra result list first
- [ ] You can explain: call → generator object → `next` / `for` → exhaustion

Stretch after Files week: `yield` from `csv.DictReader` on `seed/large_sample.csv`.

Step-by-Step Annotated Python 3 Code

Example 1 — Basic function with default and return

```
# file: utils_math.py

def calculate_bill(units, rate_per_unit=8.5, fixed_charge=50):
    """
    Calculate electricity bill.
    units: int — consumption
    rate_per_unit: float — default 8.5 INR
    fixed_charge: flat fee added every month
    """
    if units < 0:
        return None  # invalid input guard
    variable_cost = units * rate_per_unit
    total = variable_cost + fixed_charge
    return round(total, 2)

# Call with positional args
print(calculate_bill(120))           # uses default rate

# Call with keyword args (order flexible)
print(calculate_bill(units=200, rate_per_unit=10))
```

Example 2 — *args and* *kwargs

```
def build_profile(name, *skills, **meta):
    """
    name: required positional
    *skills: tuple of extra positional args (Java, Python, ...)
    **meta: dict of keyword extras (city=..., experience=...)
    """
    profile = {"name": name, "skills": list(skills)}
    profile.update(meta)
    return profile

p = build_profile("Arjun", "Python", "SQL", city="Hyderabad", experience=2)
print(p)
# {'name': 'Arjun', 'skills': ['Python', 'SQL'], 'city': 'Hyderabad', 'experience': 2}
```

Example 3 — Scope demonstration

```
counter = 0  # global

def increment():
    global counter   # needed because we ASSIGN to counter
    counter += 1
    return counter

print(increment())  # 1
print(increment())  # 2
```

Example 4 — Lambda with sorted

```
students = [
    {"name": "Meera", "score": 78},
    {"name": "Ravi", "score": 92},
    {"name": "Anita", "score": 85},
]

# Sort by score descending
ranked = sorted(students, key=lambda s: s["score"], reverse=True)
for s in ranked:
    print(s["name"], s["score"])
```

Example 5 — `map` / `filter` / `reduce`

```
from functools import reduce

marks = [55, 72, 91, 40, 88]
graded = list(map(lambda m: "Pass" if m >= 35 else "Fail", marks))
toppers = list(filter(lambda m: m >= 90, marks))
highest = reduce(lambda a, b: a if a > b else b, marks)
print(graded)
print(toppers)   # [91]
print(highest)   # 91
```

Example 6 — Timing decorator

```
import time

def timing_decorator(func):
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        print(f"[TIMER] {func.__name__}: {time.perf_counter()-start:.4f}s")
        return result
    return wrapper

@timing_decorator
def fetch_data():
    time.sleep(0.5)
    return ["row1", "row2"]
```

Example 7 — Login decorator

```
logged_in_user = None

def require_login(func):
    def wrapper(*args, **kwargs):
        if logged_in_user is None:
            print("Access denied. Please login first.")
            return None
        return func(*args, **kwargs)
    return wrapper

@require_login
```

```
def view_salary():
    print(f"Salary slip for {logged_in_user}")
```

Example 8 — Generator vs list (memory idea)

```
def squares_list(n):
    result = []
    for x in range(n):
        result.append(x * x)
    return result  # whole list exists at once

def squares_gen(n):
    for x in range(n):
        yield x * x   # one value at a time

print(squares_list(5))           # [0, 1, 4, 9, 16]
print(list(squares_gen(5)))      # same values, different delivery

g = squares_gen(3)
print(next(g))  # 0
print(next(g))  # 1
print(next(g))  # 4
```

Example 9 — Generator expression + row stream

```
names = ["Ravi", "Ananya", "Om", "Priya"]
long_names = (n for n in names if len(n) >= 5)
print(list(long_names))  # ['Ananya', 'Priya']

rows = [
    {"sku": "TEA", "qty": 2},
    {"sku": "SAMOSA", "qty": 1},
]

def read_rows(rows):
    for row in rows:
        yield row

for row in read_rows(rows):
    print(row["sku"], row["qty"])
```

Practice Exercises

Basic

1. Write `celsius_to_fahrenheit(c)` returning `F = C * 9/5 + 32`.
   Model answer:

```
def celsius_to_fahrenheit(c):
    return c * 9 / 5 + 32
```

2. Write `full_name(first, last, middle="")` returning formatted name.
   Model answer:

```
def full_name(first, last, middle=""):
    if middle:
        return f"{first} {middle} {last}"
    return f"{first} {last}"
```

3. Write `sum_all(*numbers)` returning sum of any count of numbers.
   Model answer:

```
def sum_all(*numbers):
    return sum(numbers)
```

4. Predict the output of these three scope snippets (use LEGB: Local → Enclosing → Global → Built-in):

```
# Snippet A
x = 10
def show():
    print(x)
show()
```

```
# Snippet B
x = 10
def bump():
    x = x + 1
    print(x)
bump()
```

```
# Snippet C
x = 10
def bump():
    global x
    x = x + 1
    print(x)
bump()
print(x)
```

Model answer:
A prints `10` (reads global). B raises `UnboundLocalError` (assignment makes `x` local, so the read on the right fails). C prints `11` then `11` (`global` reassigns the module name).
5. Use `map` to add 5% GST to `[100, 200, 50]` and `filter` to keep totals ≥ 150.
Model answer:

```
prices = [100, 200, 50]
with_gst = list(map(lambda p: round(p * 1.05, 2), prices))
big = list(filter(lambda p: p >= 150, with_gst))
# with_gst → [105.0, 210.0, 52.5]; big → [210.0]
```

6. Write generator `countdown(n)` that yields `n … 1`. Print with a `for` loop.
   Model answer:

```
def countdown(n):
    while n >= 1:
        yield n
```

```
        n -= 1

for v in countdown(5):
    print(v)
```

7. Using `next` twice on `countdown(5)`, what prints? Then finish with `list` on the same generator object.
   Model answer:
   `next` twice → `5` then `4`. `list(g)` on the same object → `[3, 2, 1]` (remaining only).

Intermediate

1. Refactor ATM menu into functions: `show_menu()`, `deposit()`, `withdraw()`.
   Model answer:
   Split validation into helpers; each action calls `is_positive(amount)` and updates `balance`. See lab kit `13_function_refactor`.

```
def show_menu():
    print("1 Deposit  2 Withdraw  3 Quit")

def deposit(balance, amount):
    return balance + amount

def withdraw(balance, amount):
    if amount > balance:
        raise ValueError("Insufficient funds")
    return balance - amount
```

2. Create `@log_calls` decorator printing function name and arguments.
   Model answer:

```
def log_calls(func):
    def wrapper(*args, **kwargs):
        print(f"CALL {func.__name__} args={args} kwargs={kwargs}")
        return func(*args, **kwargs)
    return wrapper
```

3. Write generator `fibonacci(n)` yielding first `n` Fibonacci numbers.
   Model answer:

```
def fibonacci(n):
    a, b = 0, 1
    for _ in range(n):
        yield a
        a, b = b, a + b
```

4. Use `functools.reduce` to find the longest name in `["Ravi", "Ananya", "Om"]`.
   Model answer:

```
from functools import reduce
names = ["Ravi", "Ananya", "Om"]
longest = reduce(lambda a, b: a if len(a) >= len(b) else b, names)
# "Ananya"
```

5. Write a generator expression for squares of even numbers from `0` to `10` (inclusive). Convert to list once to print.
   Model answer:

```
print(list(x * x for x in range(0, 11, 2)))
# [0, 4, 16, 36, 64, 100]
```

6. Implement `iter_rows(rows)` with `yield` over an in-memory list of dicts; print only the first two rows (lab pattern).
   Model answer:

```
def iter_rows(rows):
    for row in rows:
        yield row

for i, row in enumerate(iter_rows(SAMPLE_ROWS)):
    print(row)
    if i >= 1:
        break
```

Interview-style

1. Write decorator that retries a function up to 3 times on exception.
   Model answer:

```
def retry(times=3):
    def decorator(func):
        def wrapper(*args, **kwargs):
            last = None
            for _ in range(times):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    last = e
            raise last
        return wrapper
    return decorator
```

2. When choose generator over list for 10 million records?
   Model answer:
   Prefer a generator when you process records one at a time and do not need random access or the full collection in memory. A list of 10 million rows can exhaust RAM; `yield` streams values lazily and keeps peak memory low.
3. `map` / `filter` vs list comprehension — when prefer each?
   Model answer:
   Prefer a comprehension when the transform/condition is short and readability matters (most campus code). Prefer `map` / `filter` when you already have a named function to reuse, or when matching interview/library style. Always know both.
4. What is generator exhaustion? How do you iterate again?
   Model answer:
   After one full pass (`for` , `list(gen)` , or enough `next` calls), the generator raises `StopIteration` and yields nothing more. To iterate again, call the generator function (or rebuild the generator expression) to create a **new** generator object.

5. Generator expression vs list comprehension — one concrete difference besides syntax?
   Model answer:
   List comprehension is eager and reusable; a generator expression is lazy, memory-light, and single-pass. Example: `sum(x*x for x in range(1_000_000))` never builds a million-element list.

Mini-Project (Assignment): Reusable Utility Library

Title: `ksr_utils` — Personal Python Toolkit
Lab kit (build here): 15_Lab_Assets/30_ksr_utils_toolkit/
Also use: 14_decorator_login_gate, 16_venv_package_bootstrap
Generators (after this topic): 15_generator_csv_stream — not part of this homework

Homework scope: functions + lambda/`map`/`filter` + decorators. **No `yield` / generators.**

What to build

File	Functions	Notes
`validators.py`	`is_email`, `is_phone_in`, `is_positive`	Pure functions; no `print`
`formatters.py`	`format_inr`, `format_date(day, month, year)`	₹ formatting; zero-padded DD-MM-YYYY (no `datetime` module yet)
`transforms.py`	`with_gst`, `keep_high`	Use `map` and `filter` (wrap with `list(...)`)
`decorators.py`	`timing`, `require_token`	Wrapper functions; `*args` / `**kwargs`
`__init__.py`	re-exports	Clean imports: `from ksr_utils import format_inr`
`demo.py`	smoke test	Prints + one timed call + GST/filter demo

Stretch

- Add `is_strong_password(s)` (length ≥ 8, has digit).
- Add `@retry(times=3)` decorator for flaky calls.
- **After generators:** finish `generators.py` (`read_rows` must `yield`).
- **Later (Files week):** `yield` rows from a CSV with `csv.DictReader`.
- Add `README.md` with install/run and 3 import examples.

Done when

- [ ] Starter kit `demo.py` passes (**without** generators)
- [ ] `with_gst` / `keep_high` use `map` / `filter`
- [ ] Package pushed to GitHub
- [ ] README shows how a teammate imports your helpers

Interview Q&A

Shared with students after concepts and the mini-project. Use for review and interview prep — questions and model answers are included.

1. Why are functions the building blocks of clean Python code?
2. Can a function return multiple values? How?
3. What happens when you call a generator function? What is exhaustion?
4. Generator expression vs list comprehension — when prefer each?
5. When would you use `map` / `filter` instead of a list comprehension?
6. Why might a login-check decorator be useful before a sensitive function runs?
7. How does `@timing_decorator` relate to production monitoring?

Q1. What is the difference between `*args` and `**kwargs`?
A: `*args` collects extra positional arguments into a tuple; `**kwargs` collects extra keyword arguments into a dictionary. They make functions flexible when the number of inputs varies.

Q2. Explain decorators in simple terms.
A: A decorator wraps a function to extend its behavior (logging, timing, auth) without modifying the original function body. Syntax `@dec` applies `func = dec(func)`.

Q3. Generator vs list comprehension?
A: List comprehension builds the entire list in memory immediately and is reusable. A generator produces items lazily on demand, which saves memory for large or infinite sequences, but is exhausted after one full pass.

Q3b. What happens when you call a generator function?
A: The body does not run yet — you get a generator object. Values appear when you iterate (`for`) or call `next()`. After all values are taken, further `next` raises `StopIteration` (exhaustion); call the function again for a fresh generator.

Q4. What is a lambda function? When not to use it?
A: Lambda is an anonymous single-expression function. Avoid it for complex logic, multiple statements, or when readability suffers — use `def` instead.

Q5. What are `map`, `filter`, and `reduce`?
A: `map` applies a function to every item; `filter` keeps items where the function is true; `reduce` (from `functools`) folds items into one value. In Python 3, `map` / `filter` are lazy — use `list(...)` when you need a concrete list. Prefer comprehensions/ `sum` when clearer.

Q6. What is variable scope? Explain LEGB.
A: Scope determines where a name is visible. Python searches Local → Enclosing (nested functions) → Global (module) → Built-in (len, print, etc.).
